

Report for Laboratory 8: Motor Position Control

Julian Soh

Department of Mechanical Engineering, Saint Martin's University
ME/EE 477—Embedded Real-Time Computing for Mechanical Engineers

July 19, 2026

Abstract. This lab implemented a program that is a closed-loop control system whereby a motor that was being driven previously by velocity control is now modified using a position control system. A MATLAB tool was used to design an appropriate proportional-integral-derivative (PID) controller for the system.

1 Introduction

This laboratory exercise modified a previously implemented velocity control system to use position control instead. The closed-loop system is shown in Figure 1.

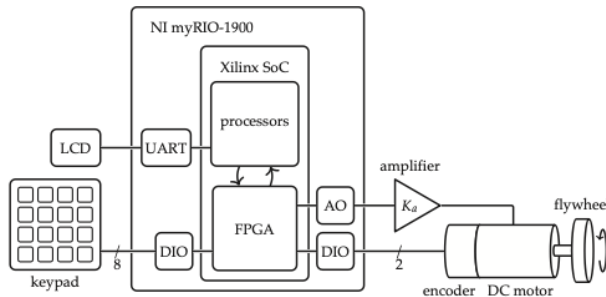


Figure 1: Schematic diagram of the target closed-loop system (Picone et al., 2024).

The objectives of this lab were:

1. To implement a position control system for an inertia-dominated load
2. To explore appropriate path planning
3. To integrate the use of a standard MATLAB design tool into the application development system

2 Materials and Methods

2.1 Materials

The materials used in this experiment were:

- Windows 10 computer with myRIO support for C and Eclipse IDE.
- NI myRIO-1900
- Keypad
- Serial display
- PMDC motor
- Analog current amplifier
- DC power supply
- Quadrature encoder
- Flywheel
- MATLAB (version R2026a Update 3 used)

2.2 PID Controller Design in MATLAB

MATLAB was used as the primary design environment to tune a proportional-integral-derivative (PID) controller suitable for the plant dynamics in this lab. The controller structure used for design and analysis is shown in Figure 2. For context, Figure 3 shows the control architecture from the previous lab (Lab 7), which was based on PI velocity control.

Comparing the two diagrams highlights the key change in this work: the PID/PIDF design introduces derivative action (with filtering) to improve transient shaping and robustness to measurement noise relative to the earlier PI-only formulation.

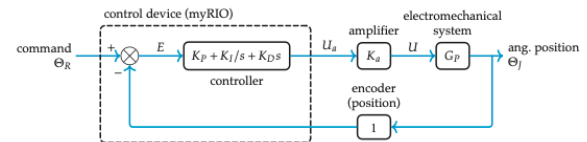


Figure 2: PID controller block diagram used for controller design (Picone et al., 2024).

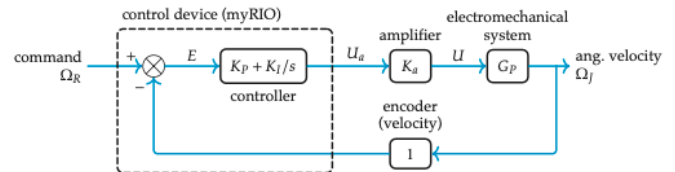


Figure 3: Control block diagram for the Lab 7 PI velocity-control system (Picone et al., 2024).

2.3 MATLAB Scripts

Two MATLAB scripts were developed for this portion of the lab: one for PID-family controller synthesis and one for performance evaluation against reference and model-based responses.

2.3.1 PID Controller Script

The first script was used to tune a PIDF compensator with MATLAB's Control System Toolbox using `pidtune()`. The design target was closed-loop reference tracking with an approximate control bandwidth of 10 Hz. A derivative-filtered PID (PIDF) form was selected to improve noise robustness by rolling off the high-frequency gain of the derivative action.

After obtaining the continuous-time controller model, the script converted it to a discrete-time implementation using `tf()`, `c2d()`, and `tfdata()` with sampling time $T = 0.005$ s. The resulting discrete transfer function was then decomposed into second-order sections with `tf2sos()` to produce biquad realizations. Finally, `sos2header()` (Appendix E.1) exported those biquad coefficients to the C header file `myPIDF.h`, which was subsequently included by the program's `main()` routine.

2.3.2 Performance Analysis Script

The second script loaded the measured position-control response from `Lab8.mat` and compared it with two benchmarks: the ideal reference displacement trajectory and the response predicted by the dynamic model.

3 Laboratory Exploration and Evaluation

3.1 Problem L8.1

Write, test, and debug the program described in section L8.3.

The program can be found at https://github.com/juliansoh/ME477/blob/main/workspace/Julian_lab8/main.c.

The program follows the same overall architecture used in Lab 7: a Main thread handled setup and user interaction through `ctable2()`, while a Timer thread handled interrupt-timed control execution and data logging. In this lab, the active controller definition was taken from the header generated by the MATLAB design script.

L8.3.1 Two Threads. Main Thread. The Main thread was responsible for initialization, thread setup, and operator interaction. Its flow was:

1. Initialize table-editor variables.
2. Initialize path-profile variables for ramp generation.
3. Configure the timer IRQ source (consistent with Labs 6 and 7).
4. Register and create the Timer thread, passing pointers to both the table data and the path profile through the thread-resource structure.
5. Launch the table editor with three display-only fields:
 - `P_ref`: revs
 - `P_act`: revs
 - `VDAout`: mV
6. On table-editor exit, clear the Timer-thread ready flag and wait for clean thread termination.

A representative resource structure was:

```
typedef struct {
1  NiFpga_IrqContext irqContext; // context
2  table *a_table; // table
3  seg *profile; // profile
4  int nseg; // no. of segs
5  NiFpga_Bool irqThreadRdy; // ready flag
6  ThreadResource;
7 }
```

Timer Thread. The Timer thread called the ISR-side logic each sample period. At thread start, shorthand pointers were created for table channels and ramp data, for example:

```
double *pref = &((threadResource->a_table + 0)->
1 value);
double *pact = &((threadResource->a_table + 1)->
2 value);
double *VDAmV = &((threadResource->a_table + 2)->
3 value);
4 seg *mySegs = threadResource->profile;
5 int nseg = threadResource->nseg;
```

The thread ran in an IRQ-paced loop and exited only when its ready flag was cleared. Before entering the loop, it initialized analog I/O, forced the motor command to zero with `Aio_Write()`, and configured the encoder counter interface.

At each interrupt cycle, it performed the following:

1. Arm the next interval by waiting on IRQ assertion, writing the Timer Write Register, and writing TRUE to the Timer Set Time Register.
2. Call `Sramps()` to compute the current reference position Θ_R .
3. Call `pos()` to measure motor position Θ_J .
4. Compute control error as $e = \Theta_R - \Theta_J$.
5. Call `cascade()` with the PIDF filter to compute control voltage, then clamp the command to $[-10.0, +10.0]$ V.
6. Write the command to DAC channel AOC0 via `Aio_Write()`.
7. Update table display values to reflect current controller state.
8. Log current-sample data for later analysis.

L8.4 Functions. Three functions were required inside the ISR loop:

- `cascade()`: reused the Lab 6 biquad difference-equation implementation (double-precision arithmetic throughout). For this application, one biquad section was used.
- `pos()`: read the encoder counter and returned displacement as double in BDI (encoder counts), referenced to the first recorded position.
- `Sramps()`: generated the reference trajectory Θ_R from a segment list, advancing one step per control cycle and repeating the full path continuously.

The path profile was initialized in `main()` and passed to the Timer thread using pointers in the shared resource structure. Access to the segment type required:

```
#include "sramps.h"
```

```

3 typedef struct {
4     double xfa; // position
5     double v;   // velocity limit
6     double a;   // acceleration limit
7     double d;   // dwell time (s)
8 } seg;

```

For Lab 8 testing, the profile used eight ramp segments:

```

1 double vmax = 50.0;
2 double amax = 20.0;
3 double dwell = 1.0;
4
5 seg mySegs[8] = {
6     {10.125, vmax, amax, dwell},
7     {20.250, vmax, amax, dwell},
8     {30.375, vmax, amax, dwell},
9     {40.500, vmax, amax, dwell},
10    {30.625, vmax, amax, dwell},
11    {20.750, vmax, amax, dwell},
12    {10.875, vmax, amax, dwell},
13    { 0.000, vmax, amax, dwell}
14 };
15 int nseg = 8;

```

This sequence produced four equal upward ramps (each 10.125 rev), followed by four downward ramps that returned the shaft to its starting position. All segments shared the same velocity and acceleration limits, with a 1 s dwell at each endpoint before progressing to the next segment.

The collected data were then opened in MATLAB from the Lab8.mat file, and the resulting plot is shown in Figure 4.

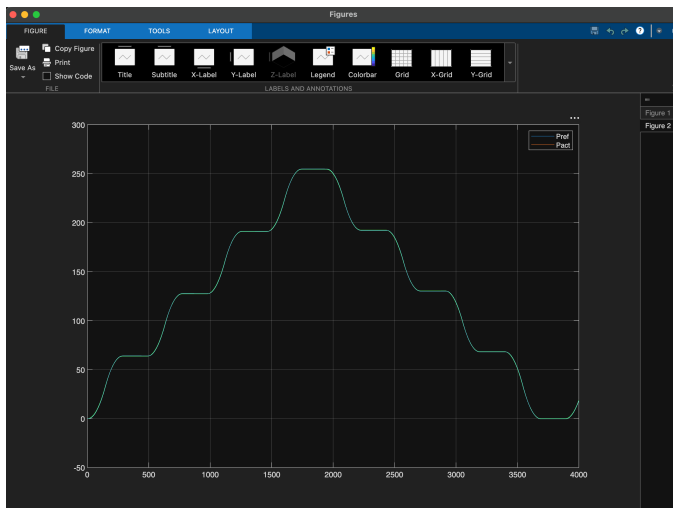


Figure 4: MATLAB plot generated from the collected data stored in Lab8.mat.

3.2 Problem L8.2

Write a MATLAB script (not the one in which you designed the controller) to analyze the result of running the program that you wrote in lab problem L8.1. It should do the following.

1. Load the experimental results from the Lab8.mat file.
2. Define a discrete version of the motor/load plant transfer from lab exercise 7. Consider using `c2d()`.
3. Form the discrete controller from the values in the PIDF array in Lab8.mat.

4. Form the closed-loop system models relating the reference position Θ_R input to the position Θ_J and torque T outputs:

$$\frac{\Theta_J(z)}{\Theta_R(z)} = G_1(z), \quad \frac{T(z)}{\Theta_R(z)} = G_2(z).$$

5. Using `lsim()`, simulate the system to find the theoretical responses for both the position $\Theta_J(t)$ and the torque $T(t)$ to the reference position $\Theta_R(t)$ array that you stored in Lab8.mat.
6. In a single MATLAB figure, plot the results in three subplot(s) versus time, as follows:
 - (a) Reference position, theoretical position, and experimental position.
 - (b) Experimental error (reference-experimental position) and theoretical error (reference-theoretical position).
 - (c) Theoretical and experimental torque.

3.3 Problem L7.3

Write, test, and debug the program described in Section L7.2. For debugging purposes, use the base set of parameters.

The PI motor velocity controller described in Section L7.2 was implemented in C and tested on the myRIO platform using the base controller gains $K_P = 0.104$ V/s/rad and $K_I = 2.07$ V/rad. Several debugging steps were required to verify proper operation of the velocity measurement, controller calculations, and actuator output. After correcting implementation issues, the controller successfully regulated motor speed. During testing, the measured velocity converged to approximately the commanded speed and the control output remained bounded. As shown in Figure ?? (Figure 3), the final implementation exhibited stable closed-loop operation and was used for the remaining laboratory exercises.

A video of this steady-state experiment could be found at <https://github.com/juliansoh/ME477/blob/main/Reports/Lab7/videos/Steady-state.mp4>.

The program implementation is available at https://github.com/juliansoh/ME477/blob/main/workspace/Julian_lab7/main.c.

4 Author Contributions

There was only one author for this report and lab.

References

Rico A. R. Picone, Joseph L. Garbini, and Cameron N. Devine. *An introduction to real-time computing for mechanical engineers*. The MIT Press, 2024.